

Homework 2 - CSCI633

Evan Lin

03/05/2026

Question 1.

1) Name and explain three possible perspectives from which one can analyze metaheuristic optimization algorithms.

(a) **Algorithm as an Iterative Process:** This first perspective views an algorithm as a mathematical procedure that repeatedly runs to generate new solutions from the current ones. Starting from an initial guess, the algorithm iteratively updates solutions until it converges to an optimum. This helps analyze convergence rates and how quickly an algorithm finds solutions.

(b) **Algorithm as a Self-Organization System :** This second perspective views algorithms as systems that evolve toward ordered states through interactions among components. Key elements include diversity, selection pressure, and evolution over time. This helps understand how algorithms organize themselves to find good solutions without external help.

(c) **Exploration and Exploitation:** This last perspective analyzes how algorithms balance two key components which are exploration and exploitation. Too much exploration leads to slow convergence while too much exploitation leads to getting stuck in local optima.

2) What is the intensification-diversification trade-off?

Intensification, also known as exploitation, is a local search that focuses on finding the local optimum. This leads to fast convergence but at the risk of getting stuck in local optima.

Diversification or exploration is a global search that explores new regions for promising areas. It avoids local optima but converges slowly.

The **intensification-diversification trade-off** is the balance between these two components. Like mentioned previously, too much intensification causes premature convergence while too much diversification wastes computational effort. So finding the balance is crucial for algorithm performance.

For an **example**, searching for the tallest mountain in a range. Diversification would be like flying over the entire range to spot peaks while climbing a specific peak would be intensification. Using both to find the highest peak will be more efficient than just using one.

Question 2.

List and properly explain the primary features of a meta-heuristic optimization process based on evolution, e.g., a genetic algorithm.

The three primary features are:

- **Crossover:** The recombination of two parents by giving parts to offspring. This feature provides a local search within a subspace and helps the system converge. In genetic algorithms, crossover typically has a high probability like 60%+.
- **Mutation:** The random alteration of parts of a solution to generate new genetic characteristics. Mutation provides the main mechanism for global exploration and helps maintain diversity. It prevents the algorithm from getting stuck in local optima. In genetic algorithms, mutation typically has a low probability like 5% or less.
- **Selection:** The survival of the fittest, where higher-quality solutions are more likely to pass their characteristics to future generations. Selection provides the driving force for convergence toward optimal solutions. Elitism, where the best solutions are guaranteed to survive, is a common form of selection.

The combination of these three features allows evolutionary algorithms to balance exploration, through mutation, and exploitation, through crossover and selection, enabling them to find optimized solution.

Question 3.

1) What is differential evolution?

Differential Evolution is a population-based stochastic optimization algorithm where it uses mutation, crossover, and selection operators like genetic algorithm. However, a key difference is that it creates new solutions by adding the weighted difference between two population vectors to a third one.

$$\mathbf{v}_i = \mathbf{x}_{r1} + F \cdot (\mathbf{x}_{r2} - \mathbf{x}_{r3})$$

where F is the mutation factor and $r1, r2, r3$ are random indices.

2) How is it different from a genetic algorithm and what benefits does it offer?

The main differences are:

- **Mutation:** Genetic Algorithm makes small random changes with low probability and Differential Evolution adds the weighted difference between two other solutions.
- **Crossover:** Genetic Algorithm mixes bits from two parents while Differential Evolution ensures each new solution inherits at least one component from the mutant vector.
- **Benefits:** Differential Evolution is simpler to implement, has fewer parameters, and often converges faster while maintaining diversity.

3) How is different from stochastic hill-climbing with random restarts?

- Differential Evolution uses multiple solutions at once while Hill-climbing uses one solution per restart.
- Differential Evolution uses differences between population members to guide search while Hill-climbing only uses local information.
- Hill-climbing relies on random restarts for exploration. Differential Evolution mutation naturally balances exploration and exploitation.
- Differential Evolution evolves smoothly as a population and Hill-climbing produces independent runs that may find different local optima.

Question 4.

(a) Rosenbrock

Table:

Temp	$\alpha = 0.1$	$\alpha = 0.5$	$\alpha = 1.0$
$T_0 = 100$	$0.00005907 \pm 0.00006537$	$0.00102909 \pm 0.00111836$	$0.00338916 \pm 0.00245397$
$T_0 = 50$	$0.00004077 \pm 0.00003548$	$0.00135939 \pm 0.00130279$	$0.00395187 \pm 0.00281383$
$T_0 = 10$	$0.00005882 \pm 0.00005320$	$0.00147241 \pm 0.00162272$	$0.00541667 \pm 0.00339616$
$T_0 = 5$	$0.00004802 \pm 0.00004989$	$0.00155656 \pm 0.00158111$	$0.00445178 \pm 0.00377390$
$T_0 = 1$	$0.00003760 \pm 0.00004774$	$0.00116627 \pm 0.00110226$	$0.00490385 \pm 0.00496641$

Discussion: The experiments show that step size α significantly impacts performance more than initial temperature. With $\alpha = 0.1$, the algorithm consistently found solutions close to the global minimum. At $\alpha = 0.5$, solution worsened, and at $\alpha = 1.0$, performance was the worst with the highest standard deviations. Initial temperature T_0 had minimal effect, with all values performing similarly for each α .

(b) Ackley

Table:

Temp	$\alpha = 0.1$	$\alpha = 0.5$	$\alpha = 1.0$
$T_0 = 100$	$18.78205806 \pm 2.72461865$	$12.52292955 \pm 9.61525349$	$4.71807935 \pm 8.44941350$
$T_0 = 50$	$18.79293376 \pm 2.72051515$	$11.89059618 \pm 9.82673020$	$2.16507338 \pm 6.01223694$
$T_0 = 10$	$18.75282488 \pm 2.95493919$	$12.45922670 \pm 9.62160598$	$4.14635108 \pm 7.95908581$
$T_0 = 5$	$18.79288699 \pm 2.72827892$	$13.07004048 \pm 9.39452463$	$3.32959057 \pm 7.28789900$
$T_0 = 1$	$18.80804376 \pm 2.68634564$	$13.78594465 \pm 9.14858298$	$4.04420393 \pm 8.02209175$

Discussion: Unlike Rosenbrock, Ackley’s function performed best with larger step sizes. With $\alpha = 0.1$, the algorithm got stuck in local minima where mean is approximately 18.8, unable to escape due to small steps. At $\alpha = 0.5$, performance improved. The best results came from $\alpha = 1.0$ and $T_0 = 50$, achieving a mean of 2.165. This shows that Ackley’s many local minima require larger steps to escape and higher initial temperatures for sufficient exploration.